Programación Orientada a Objetos en C++: Juego de Naves en Consola (class Nave) — Actividad 8

Federico Mattia, Lucas Del Pino, Maximo Orue y Ulises Ferrari
E.E.S.T. N° 1 Eduardo Ader — Vicente López
Curso: Laboratorio de Programación 5° 3° A-B
Profesor: Mansilla Muñoz York E. y Ganduglia Yamil De Jesus
Fecha: 21/10/2026

Introducción

El presente informe documenta la Actividad 8 de la materia Laboratorio de Programación (LPR), correspondiente al 2° cuatrimestre de 2026. El objetivo fue pasar del `struct` a la Programación Orientada a Objetos (POO) mediante el desarrollo de un simulador de combate espacial en consola, definiendo la clase `Nave` con atributos privados y métodos públicos que aplican el encapsulamiento. Además, se implementó una funcionalidad extra (sistema de puntaje) y se elaboró la documentación técnica: manual del programador y manual del usuario. El trabajo se realizó en grupo y el código se encuentra publicado en los repositorios de GitHub y GitLab detallados en la sección Referencias.

Desarrollo

Integrantes y roles

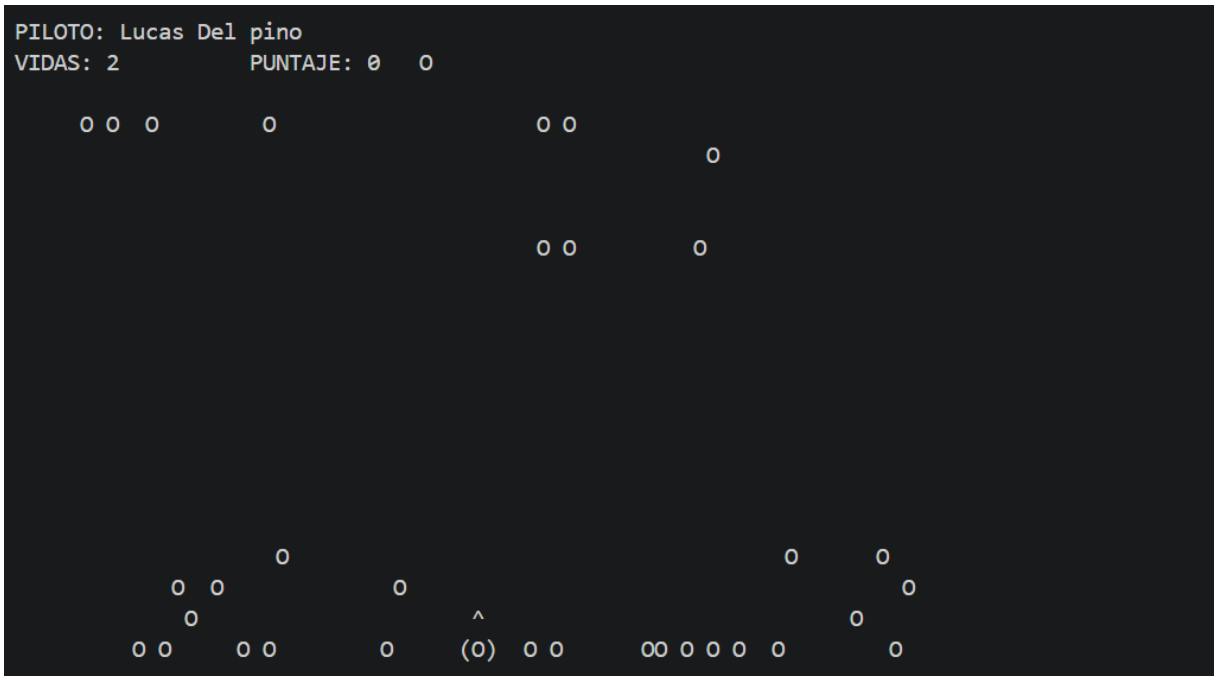
Integrante	Rol en el proyecto
Federico Mattia	Coordinador del grupo y redactor del informe APA; responsable de los repositorios.
Lucas Del Pino	Programador principal: clase Nave y sistema de puntaje.
Maximo Orue	Programador: clases Asteroide y Bala, y lógica de colisiones.
Ulises Ferrari	Documentación y pruebas: manual del programador, manual del usuario y cuestionario técnico.

Registro del código y ejecución

El código fuente se guardó en `src/main.cpp`, se compila con `g++ src/main.cpp -o src/juegodenaves.exe` y se ejecuta con `.\src\juegodenaves.exe` en Windows. Los controles son W, A, S, D para mover y ESPACIO para disparar. La Figura 1 muestra la consola con el nombre del piloto en el encabezado.

Figura 1

Consola del juego con el nombre del piloto en el encabezado



Sección private y public

En la clase `Nave`, la sección `private:` contiene los datos (`x`, `y`, `vidas`, `puntaje`) y termina donde empieza `public:`, que contiene el constructor y los métodos (`getX`, `getY`, `getVidas`, `reducirVida`, `sumarPuntos`, `getPuntaje`, `pintar`, `borrar`, `mover`). Las clases `Asteroide` y `Bala` siguen el mismo esquema.

Cuestionario técnico de control

Pregunta 1: struct vs class

¿Cuál es la principal diferencia técnica entre un struct y una clase, especialmente respecto a la sección private y public?

En C++ ambos pueden tener atributos y métodos; la diferencia técnica es el acceso por defecto: en un `struct` los miembros son públicos y en una `class` son privados. Por eso, con un struct cualquier parte del programa puede leer o modificar sus campos directamente (por ejemplo `miEntidad.id = 5`), mientras que en una clase los atributos declarados bajo `private:` solo pueden usarse desde los métodos de la propia clase, y el exterior accede únicamente por los métodos de la sección `public:`.

Pregunta 2: Encapsulamiento

¿Por qué es peligroso dejar la variable salud como pública? ¿Qué desastre podría causar un programador novato desde el main()?

En este código el atributo equivalente a la salud es `vidas`. Si fuera público, cualquier línea del `main()` podría asignarle un valor inválido, como `jugador.vidas = -50;` o `jugador.vidas = 999999;`, rompiendo las reglas del juego. Un error típico de novato es escribir `if (jugador.vidas = 0)` con un solo igual: en vez de comparar, asigna 0 y la nave muere sin motivo. Con `vidas` privado, ese código ni siquiera compila, y el método `reducirVida()` controla que el valor nunca baje de 0.

Pregunta 3: Constructor

Explicá con tus palabras para qué sirve el bloque Nave(string _nombre, ...) dentro del código.

El constructor es un método especial que tiene el mismo nombre que la clase, no devuelve valor y se ejecuta automáticamente al crear un objeto. Sirve para dejar al objeto en un estado inicial válido. En esta versión del código el constructor es `Nave(int _x, int _y, int _vidas)` (no recibe un nombre): al hacer `Nave jugador(35, 20, 3);` guarda la posición inicial (35, 20) y las 3 vidas, y además pone el `puntaje` en 0, de modo que ningún atributo queda con basura de memoria.

Pregunta 4: Desafío funcionalidad

Explicá con tus palabras qué funcionalidad elegiste agregar y en qué partes del código tuviste que insertarla.

Agregamos un **sistema de puntaje**: el jugador suma 10 puntos por cada asteroide destruido. Lo insertamos en seis lugares: (1) el atributo privado `int puntaje` en la clase `Nave`; (2) su inicialización en 0 dentro del constructor; (3) los métodos públicos `sumarPuntos(int)`, que solo acepta valores positivos, y `getPuntaje()`; (4) la línea `PUNTAJE` en el HUD, dentro del bucle principal; (5) la llamada `jugador.sumarPuntos(10)` en el bloque de colisión bala-asteroide; y (6) la línea `PUNTAJE FINAL` en la pantalla de fin de juego. Complementariamente, protegimos `reducirVida()` para que `vidas` no sea negativa y agregamos `delete` para liberar los asteroides reemplazados y la memoria al finalizar.

Conclusión

La actividad nos permitió comprender que una clase es la evolución natural del struct: agrupa datos y funciones y protege sus atributos con private y métodos públicos. El encapsulamiento evita valores inválidos y errores de programación, y el constructor garantiza un estado inicial correcto. La funcionalidad de puntaje mostró cómo ampliar una clase sin exponer sus datos, y la redacción de los manuales reforzó la importancia de documentar el software. Además, repartir los roles dentro del grupo y trabajar con repositorios compartidos nos ayudó a organizarnos mejor.

Referencias

cppreference.com. (s. f.). *Classes*. https://en.cppreference.com/w/cpp/language/classes

Mattia, F., Del Pino, L., Orue, M. y Ferrari, U. (2026). *juegodenaves* [Repositorio de código]. GitHub. https://github.com/mattia4567/juegodenaves.git

Mattia, F., Del Pino, L., Orue, M. y Ferrari, U. (2026). *juegodenaves* [Repositorio de código]. GitLab. https://gitlab.com/Masterpuchi23/juegodenaves.git

Stroustrup, B. (2013). *The C++ programming language* (4.ª ed.). Addison-Wesley.